



Time-aware and interpretable predictive monitoring system for Anti-Money Laundering

Pavlo Tertychnyi^{a,*}, Mariia Godgildieva^a, Marlon Dumas^a, Madis Ollikainen^b

^a University of Tartu, Ulikooli 18, Tartu, 50090, Estonia

^b Tallinn University of Technology, Ehitajate tee 5, Tallinn, 19086, Estonia

ARTICLE INFO

Keywords:

Anti-Money Laundering
Machine learning
Predictive monitoring

ABSTRACT

Money laundering is a global threat to society nowadays. Governments and governmental authorities fight money laundering, in part, by regulating banks and financial institutions. Financial institutions, in turn, are obligated to implement mechanisms to prevent money laundering. Usually, these prevention mechanisms include automated monitoring systems. In this paper, we propose an Anti-Money Laundering monitoring system based on machine learning techniques, which addresses three requirements: (i) generating accurate and non-redundant alerts; (ii) generating timely alerts; and (iii) associating explanations and risk estimates to each alert. The first requirement is addressed by training a machine learning classification model on different snapshots of customers' history and then feeding the classification scores generated by this model into an alerting policy designed to prevent redundant alerts. The second requirement is addressed via custom metrics for assessing the performance of classification models, which take into account the timeliness requirements that arise in the Anti-Money Laundering domain. Finally, the third requirement is addressed by applying a method for class probability calibration and by an interpretability layer based on Shapley values. The proposed monitoring system has been designed based on requirements provided by an investigation unit at a financial institution and evaluated using real-life data as well as multiple rounds of feedback from specialized subject-matter experts.

1. Introduction

Money laundering is the practice of deliberately hiding the source of acquired money from financial authorities. Generally speaking, money laundering consists of three steps: placement of illegally gained money into the financial system, layering of placed money within the financial system to obscure its origin, and integration of the cleaned money into apparently legitimate financial assets (FCE Network, 2020).

Driven by regulatory requirements, financial institutions generally deploy a variety of monitoring mechanisms to detect money laundering activity across the above three phases (placement, layering and integration) so as to report it to the relevant financial authorities. In order to automate the detection of potential money laundering activity, financial institutions make use of various tools, including Machine Learning (ML)-based monitoring systems. Such systems raise alerts whenever the behaviour of a customer suggests possibly illicit activity.

The usability of an ML-based monitoring system for Anti-Money Laundering (AML) is defined by business users and underpinned by at least three requirements. First and foremost, the monitoring system has to generate alerts that are likely to lead to identifying illicit behaviour that has not been previously detected. This requirement entails two sub-requirements. First, the monitoring system needs to produce accurate

alerts. A high number of false positive alerts will put a high workload on subject-matter experts (i.e. the investigators) who must manually check each alert. A high number of false negatives, in turn, makes the monitoring system useless since it misses most of the illicit activity. A second sub-requirement is that the system should not generate alerts over and over again for the same customer, if the new alert does not bring any additional information over previous alerts. Redundant alerts consume valuable time from the investigators.

Second, a monitoring system needs to generate timely alerts. Indeed, the earlier the money laundering activity is detected the sooner a financial institution will be able to take corresponding mitigation actions, such as reporting a case to the authorities, place limits on product usage, etc. Late alerts may bring monetary and reputational losses for the financial institution, while premature alerts may result in insufficient evidence of money laundering to proceed with an investigation. Timeliness requirement needs to be considered in the assessment of the quality of an ML monitoring system. This requires us to adjust the way the quality of the ML-based monitoring system is evaluated, as standard approaches for assessing the quality of ML models do not take time aspect into account.

* Corresponding author.

E-mail addresses: pavel.tertychnyi@gmail.com (P. Tertychnyi), mariagodg@gmail.com (M. Godgildieva), marlon.dumas@ut.ee (M. Dumas), madisollikainen@gmail.com (M. Ollikainen).

<https://doi.org/10.1016/j.mlwa.2022.100306>

Received 2 December 2021; Received in revised form 7 February 2022; Accepted 18 April 2022

Available online 20 April 2022

2666-8270/© 2022 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Third, an ML-based monitoring system needs to produce interpretable alerts, so that the investigators can determine which alerts are worth further investigation and in what direction that investigation should proceed. Well-explained alerts shorten the time needed to investigate an alert, and hence allow more alerts to be checked. Related to the above, an ML-based monitoring system for AML needs to provide accurate (calibrated) estimates of the probability that a given customer engages in illicit behaviour, so that investigators can use this information to assess the related risks.

In this paper, we present a case study involving the design and development of an ML-based monitoring system for AML in a multinational financial institution. The purpose of the research is to design and assess the performance of the monitoring system as well as to make the system usable for the end users. The proposed monitoring system addresses three key requirements: (i) generating accurate and non-redundant alerts; (ii) generating timely alerts; and (iii) associating explanations and risk estimates to each alert. The first requirement is addressed by an ML classification model that was trained on customers' financial behaviour history. An alerting policy designed to prevent redundant alerts was built on top of the classification scores generated by this ML model. The second requirement is addressed by designing custom metrics for assessing the performance of classification models, which take into account the timeliness requirements. Finally, the third requirement is addressed by an interpretability layer based on Shapley values (Shapley, 1953) and by applying a method for class probability calibration.

The proposed monitoring system has been designed based on requirements provided by an investigation unit at the financial institution and evaluated using real-life data as well as multiple rounds of feedback from specialized subject-matter experts.

The rest of the paper is organized as follows. Section 2 briefly reviews the relevant literature. Section 3 describes the proposed ML-based monitoring system for AML. Section 4 discusses an evaluation of the proposed system undertaken in a real-life environment and summarizes feedback given by subject-matter experts in this setting. Finally, Section 5 sketches the directions of the future research and summarizes our main conclusions and contributions.

2. Related work

An ML-based monitoring system is a system that uses an ML model to periodically classify entities into positive and negative classes, for example, to classify entities into potentially illicit or non-illicit. In general, ML-based monitoring systems are designed to raise alerts when an entity falls under a negative class, so that domain experts can further investigate and/or act upon such alerts. Monitoring systems are common in various domains, including equipment failure monitoring, predictive machinery maintenance, card fraud detection, etc. ML-based monitoring systems are particularly useful in domains where manual monitoring is infeasible and where it is not possible to codify all possible behaviours that may lead to an alert in the form of a rule-set. This is the case in the context of money laundering detection, as there may be millions of transactions per day (not suitable for manual classification) and where it is not possible to characterize all possible behaviours of interest via a set of rules.

The problem of designing and developing ML-based monitoring systems in the AML domain has been addressed in several previous research studies. Ketenci et al. (2021) tackled the problem of money laundering detection as a problem with time factor by proposing a novel set of tailor-made time-aware features based on time-frequency analysis. The detection model is tested on real-life data of 6680 Akbank customers, 26% of which have been engaged into money laundering activity. This ratio is very far from a real-life distribution, where the ratio of suspicious customers is usually very low. Thus, in typical real scenarios, an AML monitoring system needs to handle datasets with strong class imbalance. Although the work of Ketenci et al. provides

a solution for money laundering detection, it does not tackle the problem of determining the time when the output of the classification model should lead to an alert. It also does not address the question of explaining the predictions made by the model to end users.

ML-based monitoring techniques have also been applied for money laundering monitoring by Chao et al. (2019). The authors monitored a capital flow to tackle trade-based money laundering using Bayesian networks. The proposed Bayes net is represented by a directed acyclic graph and a conditional probability table which are built on a training data. The evaluation of the developed tool has been done by using ROC-AUC, F-score and Geomean measure, without considering the timing of the prediction. Monitoring techniques were also used to detect cellular fraud in a work which was presented by Fawcett and Provost (1997). The authors have developed the rule-based model for alert generation to detect cellular cloning fraud. Their research has been performed on a database of call records and model performance was measured as accuracy averaged by time of model runs. As an additional performance metric, they calculate the cost of alerts. While this latter study considers the cost of false alerts (i.e. false positives), it does not take into account the cost of generating an alert too early or too late (timeliness of alerts).

From the perspective of ML algorithms, there is a great variety of both supervised and unsupervised approaches used. Jullum et al. (2020) studied the usage of gradient boosting trees in the implementation of XGBoost framework to detect suspicious transactions on real-life data of Norway's bank DNB. The performance of the proposed method was evaluated using ROC-AUC, Brier Score and proportion of positive predictions. Ahmed (2021) made comparisons of Random Forest (RF) and XGBoost algorithms on three public datasets of suspicious transactions: Elliptic dataset, Ethereum Illegal Accounts dataset and NOAA dataset. The performance of the models was tested using precision, recall, accuracy and F1 scores. Support Vector Machines (SVM) were used for money laundering detection by multiple authors (Alkhalili et al., 2021; Raiter, 2021). Usually, SVMs are used on small and medium datasets due to time constraints required for model training, especially when the problem requires usage of non-linear kernels. This makes it practically impossible to use this family of algorithms in large scale real-life applications. As of unsupervised approaches, Mubalaika and Adali (2018) investigated the usage of Stacked AutoEncoders (SAE) and Restricted Boltzmann Machines (RBM) for real-time financial fraud detection on a synthetic dataset of mobile money transactions. Chen et al. (2021) used Autoencoders (AE) and Variational Autoencoders (VAE) for money laundering detection on real-life data from a Malaysian bank. To enhance the performance and tackle class imbalance Wasserstein Generative Adversarial Networks (WGAN) were used to generate artificial fraudulent transactions.

In a monitoring task, an essential question is how to properly evaluate the performance of the model. Traditional performance metrics do not represent the real state of a model very precisely and can highly overestimate its performance. The question of timing of alerts has been addressed by Dachraoui et al. (2015), who propose a method to balance the expected gain in the classification cost in the future and the cost of delaying the decision. The method has been tested on both synthetic data and real-life TwoLeadECG dataset. The performance of alert generation systems in the predictive maintenance domain has been assessed by Sipos et al. (2014). The authors are predicting equipment failures by mining the data from event logs of medical devices. To fulfil the timing of alerts requirement they redesigned the definition of true positives as cases where an alert occurred in a predefined interval before failure and false positives as cases where an alert occurred outside of this interval. Final model performance is measured by Predictive-Maintenance-based AUC as usual PRAUC but with custom recall and precision. Similar approach of performance metric redefinition has been chosen by Zhang et al. (2016). They also took into account problem of alerts reoccurrence in predictive maintenance task. The proposed approach was validated on two enterprise systems' log traces: web server cluster and mailer server cluster. The framework's performance has been estimated by custom PRAUC metric (made from custom-defined recall and

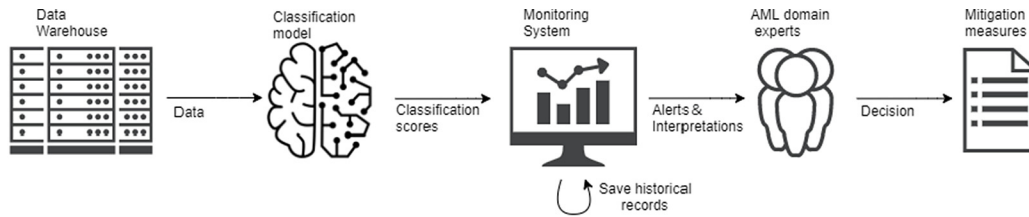


Fig. 1. Simplified view of a pipeline for ML-based customers monitoring system.

custom-defined precision), predictable interval metric and predictable frequency metric. In this paper, we take inspiration from the measures proposed in work of Sipos et al. in order to evaluate the performance of predictive models for AML monitoring.

A crucial requirement in monitoring systems, beyond the AML domain, is interpretability, since in most of the cases, the alerts generated by such systems have to be analysed by subject-matter experts. Xing et al. (2011) tackled the interpretability of models in early time-series classification task. They extracted local shapelets as features. A shapelet represents a segment of a time-series with a certain shape. The results of the algorithms were evaluated on seven datasets from the UCR time series archive. A similar approach based on shapelets was considered by He et al. (2015). This latter approach has been tested on two synthetic datasets and real-world Wafer and ECG datasets. Both He et al. (2015) and Xing et al. (2011) are focused on design of interpretable features and early classification problem, i.e. timeliness. But neither of them have defined the monitoring setup with the algorithm of alerts raising and assessment of alerts' quality. The interpretability of models was not tested on real end users.

3. Proposed approach

In this section, we present the design of an ML-based monitoring system that addresses the three requirements spelt out in Section 1, namely: (i) accuracy and non-redundancy of alerts; (ii) timeliness of alerts; and (iii) interpretability of alerts.

The simplified view of a pipeline for ML-based customers monitoring system is presented in Fig. 1. Like any other ML-based system, the first component of the system is a data extraction and pre-processing component, which extracts data from a warehouse in order to construct feature matrices that are then used to train a classification model. This classification model produces as output classification scores for every entity at any point in time (e.g. every week). The classification scores are then used by the monitoring system itself, which implements a policy for turning the classification model results into alerts. This policy takes into account previous alerts and classification scores (to prevent redundant alerts) and is calibrated to maximize certain custom classification metrics that take into account the alert timeliness requirement. The monitoring system relies on probability calibration and explainable ML techniques to provide risk scores and explanations to domain experts, together with each generated alert. Based on this input, the domain experts then proceed to investigate each alert and take mitigation measures when required.

The rest of this section presents the design of the monitoring system, the custom metrics to account for alert timeliness, the probability calibration and interpretability layer associated to the proposed monitoring system.

3.1. Monitoring system

The input of the proposed monitoring system (MS) is a dataset of customers and their financial activities in the financial institution (see Section 4.1 for a detailed description of the dataset). The dataset contains the history of transactions of each customer during a period of time and in this study, we handle 6 months long transaction histories. If a customer joined the financial institution less than 6 months ago,

then the customer is still monitored and treated as if it was dormant at the beginning of 6 months period. The dataset includes two groups of customers:

1. Customers who were reported (herein called "Reported") to the relevant authority, such as FIU (Financial Intelligence Unit), because of a detected potentially illicit activity. Reported customers have at least one report, and each report has a correspondent report date.
2. Those who were never reported. Out of the whole customer base of never reported customers, we sampled a set of customers so those we call Randomly Sampled.

The task is to monitor financial activity of the bank's customers and to raise alerts if and when potentially illicit activity is detected. A customer can perform multiple potentially illicit actions, therefore the customer can get multiple alerts. The alerts are generated on the basis of an ML classification model trained on historical data of customers' financial activity and historical labels. The problem in hand was treated as a classification problem, rather as a use case for survival analysis. The main goal, ultimately, is to find if a customer is likely to be found potentially illicit by an investigator at time T , rather than finding out whether the customer will eventually be found potentially illicit. Survival models are suitable when we wish to estimate the time until an event happens, which in this context would mean the time until a customer engages in or is found to engage in illicit behaviour. But in this paper, we rather focus on generating alerts that investigators would find useful.

In our approach, customers are classified on the basis of a range of profile features as well as their history of transactions during a given period (e.g. 6 months). This approach uses two predictive models chained one after the other. The first predictive model is a logistic regression model designed to filter out customers who clearly do not exhibit behaviour similar to that of reported customers. It is based on a small number of features that can be computed in a scalable manner, which is essential in order to be applicable to millions of customers. The second model is a tree ensemble model (using extreme gradient boosting, specifically CatBoost) and a larger number of features including aggregate features (transaction sums, counts and similar aggregates), demographic features (type of customer, geography), and features extracted from Markov Chain Models trained on series of inbound and outbound transactions, intended to capture temporal transaction patterns. Such two-layered architecture of the classification model provides better scalability, since the most computationally demanding calculations are done for a filtered set of customers. Additionally, this architecture ensures imbalances-resistance of the model by doing knowledge-driven undersampling for the second layer model, which is performing customers classification. More detailed information on data, features extraction, model selection and comparison can be found in our previous research (Tertychnyi et al., 2020).

The classification model is run periodically — in our case, once per week. In other words, the model is run weekly and after each run, customers are classified as being potentially illicit or non-illicit. In each model run, customers are classified based on their history of transactions of previous 6 months (herein called a *customer snapshot*, or a *snapshot* for short). This means that in each model run, the customer

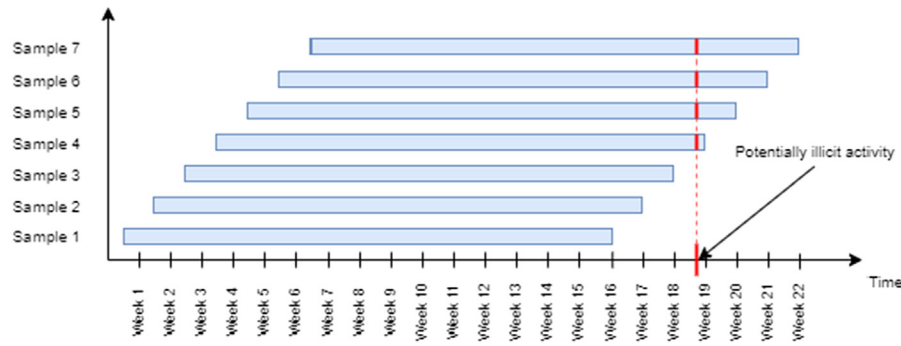


Fig. 2. Example of consequent historical snapshots of a single customer which is under monitoring.

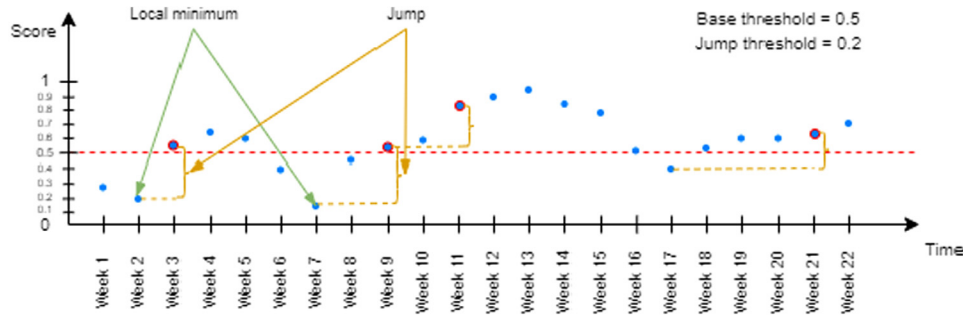


Fig. 3. Example of alerts raised for a customer. The customer is monitored for 22 weeks with weekly model runs. In this example base threshold = 0.5 and jump threshold = 0.2. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

has a week of new transactions while one week of transactions will be forgotten (see Fig. 2). In other words, the classification is made on the basis of sliding windows, to ensure that the samples are comparable. This brings a complication that in sequential monitoring we classify very similar snapshots of the same entity, with one week of new transactions and without one week of old transactions. Eventually, each customer has a sequence of classification scores which represents probabilities of being potentially illicit at each time we run the classification model.

Having a sequence of classification scores for a customer for a period of time, we have to understand when to raise alerts. These alerts will be checked manually by an AML domain expert. In light of this, it is impractical to raise alerts for the same customer every week or even several weeks in a row, because it would be likely that the alerts were due to the same set of potentially illicit activities.

Accordingly, we propose our custom MS with two parameters: (i) a base threshold – above which a customer's activity is declared to be unusual enough for an alert to be raised; and (ii) a jump threshold – a threshold of deltas between scores to detect sudden changes in a customer's activity. For an alert to be raised, corresponding classification score should meet two requirements:

1. The score has to be larger or equal to the base threshold.
2. The difference between the score and the minimum score since the last alert (later *local minimum*) should be larger than the jump threshold.

The rationale behind the first requirement is that alerts should only be raised for customers associated with a high level of risk. The second requirement, in turn, ensures that alerts are only raised when there is a change in a customer's behaviour.

Fig. 3 illustrates an example of a customer under monitoring and the corresponding alerts raised by the proposed MS. In this example, we have a customer's monitoring history of 22 weeks. Each blue dot is the probability of the customer acting illicitly which is computed by the ML classification model. The red circles are weeks when alerts

are raised for the customer. The alerts are raised on weeks 3, 9, 11 and 21. The first two weeks, the customer does not have any alerts. On week 3 the score reaches the value of 0.55 and becomes higher than the base threshold. Local minimum on week 3 equals to 0.2, so the difference between the score and local minimum is higher than the jump threshold, thus, the alert is raised on week 3. Although the scores on weeks 4 and 5 are higher than the base threshold, the scores there do not meet the second requirement and there is almost no jump in score compared with the minimum since the last alert (week 3 for the moment of monitoring at week 4), so the alerts are not raised on weeks 4 and 5. On week 9 the alert is raised again because the score has reached the level of 0.52 and exceeded the base threshold while the local minimum is at week 7 with the value of 0.18. On week 11 the alert is raised once more because the jump in score was 0.25 which is higher than the jump threshold. In other words, the behaviour of the customer has become even more unusual between weeks 10 and 11, probably due to a new unusual activity that happened. For weeks 12–16 including, alerts are not raised, again, because there was no jump in score and the score is high because of the same unusual activity detected on week 11. On week 21 the alert is raised since the difference in scores between the local minimum (week 17) and week 21 got larger than the jump threshold. The score had started to rise gradually since week 17 but reached the threshold on the week 21.

3.2. Custom metrics

To assess the quality of a MS, we rely on the notions of recall and precision typically used in the field of classification, with some adjustments to take into account the sequential setting in which the MS operates, as explained below.

3.2.1. Recall

In a one-time AML classification task, recall corresponds to the question “what is the percentage of potentially illicit customers that the AML classification model classifies as such?”. In the context of periodic AML monitoring, this question takes a temporal dimension.

Concretely, the AML classification model takes as input a snapshot of a customer's data at a particular point in time. The model then produces an output (alert or no-alert) on the basis of this snapshot, and it is later on invoked again with another (later) snapshot. This means that the AML model will classify multiple snapshots of the same customer with a timeshift. For example, if a customer is monitored for twenty weeks, and the monitoring system is run once a week, twenty samples of this customer will be labelled by the model. A naive application of recall in this setting would not be representative because, if the AML model classifies the last of those twenty samples as potentially illicit (i.e. it raises an alert) and the first 19 as non-illicit (no alert), we get the recall of $\frac{1}{20}$. Yet, the model did classify the customer as potentially illicit (i.e. an alert was raised) so arguably the recall (if we restrict the dataset to this one customer) should be 1, provided that the alert is raised at or around the time when the customer was reported as potentially illicit.

In light of the above, we adopt a definition of recall at the customer level (herein called custom recall). Specifically, we define custom recall as the number of potentially illicit customers who were alerted at least once at or around the time they were reported as potentially illicit, divided by the total number of potentially illicit customers in the dataset. To make this definition complete, we need to define the notion of "at or around the time". This is defined below as part of the definition of precision.

3.2.2. Precision

In a one-off classification setting, precision answers the question "what is the percentage of customers that the AML model classifies as potentially illicit that are indeed potentially illicit according to the ground truth?". In a periodic monitoring setting, we have to customize this definition to capture, instead, the proportion of correct alerts generated by the AML system. In other words, we need a custom notion of precision that penalizes over-alerting, in particular, repetitive alerts raised in a row for the same customer.

Accordingly, we propose a custom definition of precision that captures the idea that an alert should be raised close to the date when a customer was reported as having illicit activity. Specifically, we propose to capture the utility of an alert based on whether or not it falls under an isosceles trapezoid around a date when a report was made for a customer (see Fig. 4). Very early alerts have the utility in the amount of left leg of the trapezoid and the utility grows linearly with an alert approaching to the report date. Very late alerts have the utility in the amount of the trapezoid's right leg, and the utility decreases linearly starting from one week before the report date and ending at the point of two weeks after the report date. The lower base of the trapezoid defines what is the time range where the alerts get some positive utility; outside this range the alerts get a utility of zero. The highest utility is achieved at a time between 4 to 1 week before the report date. The rationale for this choice is that earlier alerts have more value than late alerts.

The precision is calculated as

$$precision = \frac{1}{N_A} \sum_{a \in A} \max(u_a) \quad (1)$$

where A is a set of all alerts raised for customers from the dataset, u_a is the utility of alert a calculated as the height of the trapezoid at a time of an alert a , N_A is the size of A . A customer may be reported more than one time, thus there will be a correspondent trapezoid for each of the reports, so we calculate a utility of an alert as $\max(u_a)$ if it falls under several overlapping trapezoids.

Let us see an example on Fig. 4. The customer in the example has been reported to the FIU on week 8 and got 3 alerts raised by our MS — on weeks 3, 5 and 9 with correspondent utilities of $\frac{2}{3}$, 1 and $\frac{1}{3}$ (correspondent trapezoid heights). The impact to the nominator of the precision (sum of alert utilities) of this customer would be $\frac{2}{3} + 1 + \frac{1}{3} = 2$. And impact to the denominator of the precision is +3, since there were 3 alerts generated. If the customer gets alerts only on weeks 3 and 5

then the impact to the nominator of the precision would be $\frac{2}{3} + 1 = \frac{5}{3}$, but impact to the denominator of the precision is +2, since there were only 2 alerts generated. That shows that the metric penalizes repetitive alerts raised for the same customer. As we can see such design of precision calculation also controls the timing of alerts by reducing the utility of alerts that are not raised at the correct time. The current way of calculating the precision differentiates the False Positives — a customer who does not have a report but has 1 alert raised will get a 0 total utility and the total precision will be divided by a smaller number (total number of alerts will be +1). And a customer who does not have a report but has 20 alerts raised, will still get a 0 total utility but the impact to the overall precision will be divided by a larger number (total number of alerts will be +20).

The custom metrics described above are used to find the optimal parameters of the custom MS. So, the base threshold and jump threshold are taken such that maximize F1 score on the validation set (see Section 4.3). F1 score is chosen as the optimization metric since it provides trade-off between precision and recall of the model.

$$custom_F1 = 2 \frac{custom_precision + custom_recall}{custom_precision + custom_recall} \quad (2)$$

3.3. Probability calibration

The goal of the proposed AML monitoring approach is to raise alerts when there is a high probability that a customer engages in illicit behaviour.

To this end, we need to estimate the probability that a given customer-snapshot extracted at a given point in time belongs to the illicit behaviour class. Nonlinear classification models output class probability scores that do not match to the properties of a probability distribution. For example, if the model assigns an output score of 0.8 to a set of samples, one would expect that 80% of these samples would indeed belong to the positive class. However, this is not necessarily the case for non-linear classifiers, because the classification score produced by these models represents a similarity metric and not an actual probability.

In our approach, we require that the class probabilities by the classification model should be calibrated, meaning that they are adjusted to reflect an actual probability. By ensuring this, we obtain the following benefits:

1. The model produces scores that match what an end-user would expect, thus allowing the subject-matter experts to follow a risk-based approach when tuning the parameters of the model.
2. The model is robust to periodical retraining because the output probabilities become comparable for models trained in different periods of time.

There are numerous algorithms for probability calibration. Experiments showed that in our task the best approach is beta calibration (Kull et al., 2017) (see Fig. 5 for comparison of different calibration approaches on our test data). The advantage of beta calibration is that it does not make the assumption that scores of classes in a binary classification task are normally distributed, i.e. have a distribution with infinite support. Instead, it derives a new family of calibration functions with finite support of [0,1] which are distributed according to beta distribution with

$$p(s, \alpha, \beta) = \frac{s^{\alpha-1} (1-s)^{\beta-1}}{B(\alpha, \beta)} \quad (3)$$

3.4. Explanation of predictions

The ability to assign interpretations to the predictions generated by an ML model is a common requirement in industrial applications, particularly in situations where the output of an ML model needs to be validated by a domain expert. In the context of an AML monitoring system, the alerts generated on the basis of the ML model are used to trigger investigations.

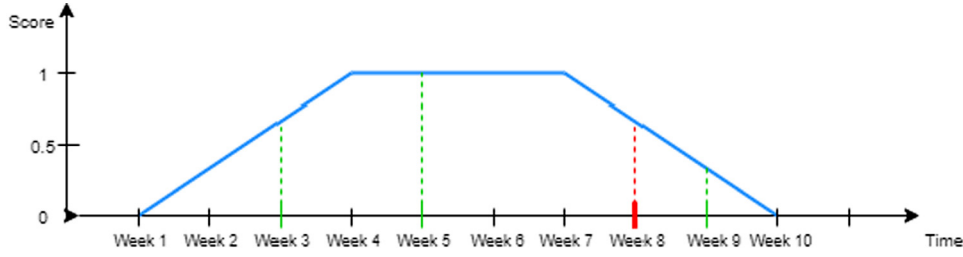


Fig. 4. Utility trapezoid for a customer. The customer has been reported on week 8 and got alerts on weeks 3, 5 and 9.

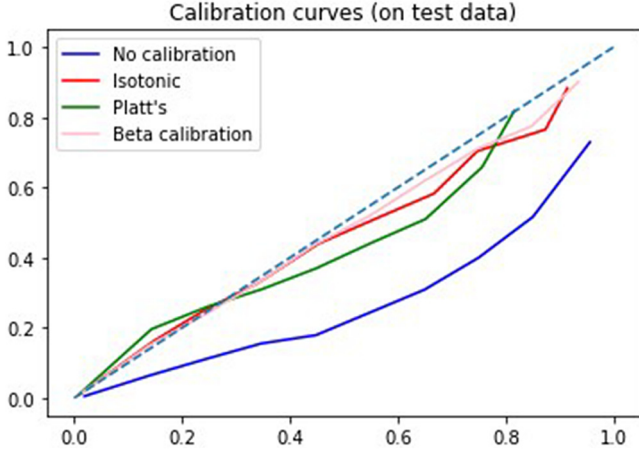


Fig. 5. Calibration curves for our model calibrated by beta calibration, isotonic calibration, Platt scaling and without calibration. Dash line is the ideal case.

Naturally, the investigators require not only an alert with a corresponding probability score, but also clues as to why this alert was produced.

To explain the alerts, we use Shapley values (Shapley, 1953). Shapley values define how to fairly distribute a payout among contributors in a coalitional game according to their contributions. In the case of an ML model, the contributors are features we feed to a classification model and the payout is the final classification score of an instance. The algorithm underpinning the Shapley values calculates the average of all marginal contributions of a feature by all possible coalitions. The formula for calculating Shapley value for a certain feature is:

$$f_i = \frac{1}{|N|} \sum_{S \subseteq N(i)} \frac{|S|! (|N| - |S| - 1)!}{|N|!} (v(S \cup \{i\}) - v(S)) \quad (4)$$

where N - the set of features, v - a value function (a payout function for a coalition of features), i - index of a feature, S - subset of set N . More details and intuition behind Shapley values could be found in the original paper (Shapley, 1953). Calculations of Shapley values according to the original paper are infeasible to make for a current set up since it requires to estimate the impact of every possible feature coalition, i.e. to train a separate model for it. Thus, we used SHAP (SHapley Additive exPlanations) framework (Lundberg & Lee, 2017) for calculating the approximated Shapley values, further addressed as *SHAP values*.

In our setup, we calculate SHAP values on each model run which gives us a picture of how each feature contributed to a model output score of each customer. To provide an explanation of the prediction, we select 5 features with top SHAP values — this corresponds to the first requirement of the MS (base threshold). Additionally, we select 5 features with the highest deltas of SHAP values between the time of classification and the time of the closest local minimum — this corresponds to the second requirement of the MS (jump threshold). Simply stated, we highlight the features which provided the highest contribution and the features which contributions have increased the most.

All the extracted features are grouped into several categories (e.g., features which represent the turnover, counterparty related features, cash transactions features, etc.) after which they are described with humanly readable definitions. Having identified the most contributive features, we map them to the predefined feature groups and form the textual description based on feature groups definitions. Eventually, there are 123 features that are mapped to 24 feature groups.

Note that if the alert is raised in the very first weeks after bootstrapping the MS, we do not have features with deltas of SHAP values, since there is nothing to compare the SHAP values with. In this case, we use the 5 features with the highest SHAP values.

A schematic view of the interpretation layer of our alert generation approach is given in Fig. 6. The figure shows that features capturing the cash patterns of outgoing transactions and the number of unique counterparties have the highest SHAP values, thus contribute to the predictions made by the model the most. Meanwhile, the contributions of sum and mean of the incoming payments have increased the most — on 1.0 and 2.4 points respectively (difference between SHAP values at the time of local minimum and SHAP values at the current time for those features).

4. Evaluation

In this section, we present an empirical evaluation of the proposed MS using real-life data from a multinational financial institution and feedback from AML domain experts. The aim of the evaluation is to address the following research questions:

- **RQ1:** To what extent the proposed custom MS reduces the number of redundant alerts (per customer) relative to a baseline MS that fires alerts based on a single class probability threshold?
- **RQ2:** How to correctly estimate the predictive performance of the proposed MS under a baseline configuration?
- **RQ3:** To what extent the tuning of the proposed MS improves its predictive performance?
- **RQ4:** To what extent domain experts perceive that the alerts generated by the custom MS are relevant (perceived relevance)?
- **RQ5:** To what extent domain experts perceive that the alerts generated by the custom MS are useful as a starting point for an investigation?

4.1. Dataset

To address the above research questions, we used an anonymized dataset of transaction histories extracted from the bank's database. The extracted dataset covers a subset of circa 330000 customers from three countries, across 81 weeks, and a total of more than 240 million transactions. Among the customers in the dataset, 0.4% are "reported" customers and 99.6% are "randomly sampled" (not reported) customers; 8% of corporate and 92% of private customers. All customers have a transaction history from March 2018 to April 2020. Each customer is represented with weekly shifted 6 months snapshots (see Fig. 2). If a customer did not have transactions during some week then the corresponding 6-months snapshot was not extracted since the

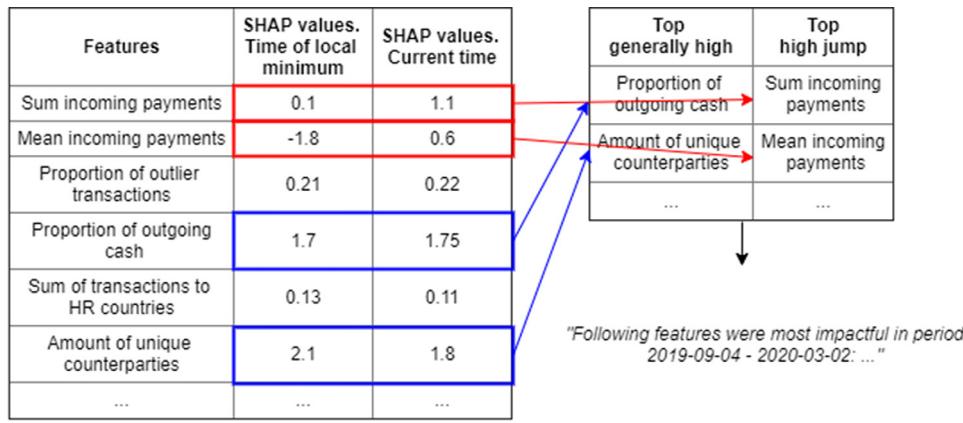


Fig. 6. Schematic example of proposed interpretations. The first step is to calculate the SHAP values for each feature as of the current time, and as of the closest time when the monitoring score reached a local minimum. The second step is to identify the 5 features with highest SHAP and the 5 features with the highest deltas of SHAP. The third step is to produce a human-readable presentation of those features.

customer was already monitored on previous iteration and no new information was added. The training period was September 2018–October 2019 (56 weeks) and testing (monitoring) period is October 2019–April 2020 (25 weeks). We use a single temporal split since this reflects the way the model would be trained and used in practice — training occurs at a given time point T based purely on data available before or at time T , and testing occurs based on data available after point T , during a certain testing period (Sheridan, 2013).

The data was used to train a classification model that predicts whether or not a customer, at a certain time point T , is engaged in activity similar to those of reported customers or not. In other words, the label used is “reported” versus “not reported” customer. The approach used to train the classification model is the one outlined in the Proposed Approach section above and described in further details in (Tertychnyi et al., 2020).

4.2. Handling of reoccurring alerts (RQ1)

The first experiment addresses RQ1 by comparing the proposed (custom) MS with a baseline (default) MS where the alerts are raised when the probability of a customer’s behaviour being illicit exceeds the predefined threshold (standard way to convert probabilities into classes). With respect to this question, the working hypothesis is that the default MS would generate periodic (weekly) alerts for several weeks in a row, most of them redundant, whereas the custom MS would generate redundant alerts, for a given customer, only sporadically. For this experiment we used a base threshold of 0.5 and a jump threshold of 0.2 for the custom MS, and a threshold of 0.5 for the default MS (using calibrated probabilities). The parameters of the utility trapezoid parameters were set as follows: beginning of the lower base – 16 weeks before the report date, end of the lower base – 8 weeks after the report date, beginning of the upper base – 6 weeks before the report date, end of the upper base – 2 weeks before the report date. This means that the alerts raised 4 months before or 2 months after the report date do not have any utility, and the alerts raised 6 weeks before to 2 weeks before the report date have the maximum possible utility. Note that if an alert was raised by the custom MS it is raised by the default MS as well, but not vice versa (see Fig. 7).

To highlight that the default MS does not handle the reoccurring alerts requirement, while the custom MS does, we plotted the distribution of number of raised alerts per customer (see Fig. 8). We can see that the custom MS raised only up to 3 alerts per customer for the testing period of 25 weeks. At the same time, the default MS raised up to 25 alerts per customer, which means that some customers were alerted on every week of monitoring.

4.3. Performance of custom MS (RQ2)

The next question is how to assess the performance of the custom MS. When performing the AML monitoring, we care not only about how many customers got alerted, but also how many alerts were raised for those customers. This means that we have to analyse two confusion matrices (CM) to understand the performance of the monitoring system:

1. **Alert level CM** – the CM where one object is one customer-snapshot. So there are multiple objects related to one customer present in the CM. CM on alert level is defined as follows:
 - TP — an alert was raised for a snapshot of a reported customer
 - FP — an alert was raised for a snapshot of a never reported customer
 - TN — an alert was not raised for a snapshot of a never reported customer
 - FN — an alert was not raised for a snapshot of a reported customer
2. **Entity level CM** – the CM where one object is one customer. So the results of model monitoring are aggregated on the entity level by the rule that if at least one snapshot of a customer created an alert (within an appropriate time frame), the customer gets a positive prediction in the entity-level CM. Specifically, the CM at the entity level is defined as follows:
 - TP – a customer was reported at least once and an MS raised at least one alert
 - FP – a customer was never reported and an MS raised at least one alert
 - TN – a customer was never reported and an MS did not raise any alert
 - FN – a customer was reported at least once and an MS did not raise any alert

Table 1 shows the entity-level and alert-level CMs for the custom MS on a test set of 587 reported and 97911 randomly sampled customers over 25 weeks of monitoring.

In the answer to RQ1 it was shown that custom MS handles reoccurring alerts and there were 3 alerts raised for a single customer at most for the test monitoring period of 25 weeks. Thus, from the CMs above, it can be noticed that there were 381 alerts raised (alert level TP) on 298 reported customers (entity level TP). And at the same time, there are 9856 snapshots of reported customers where alerts were not raised (alert level FN) and 289 reported customers that did not get any alert (entity level FN). We note that the standard alert level recall (calculated

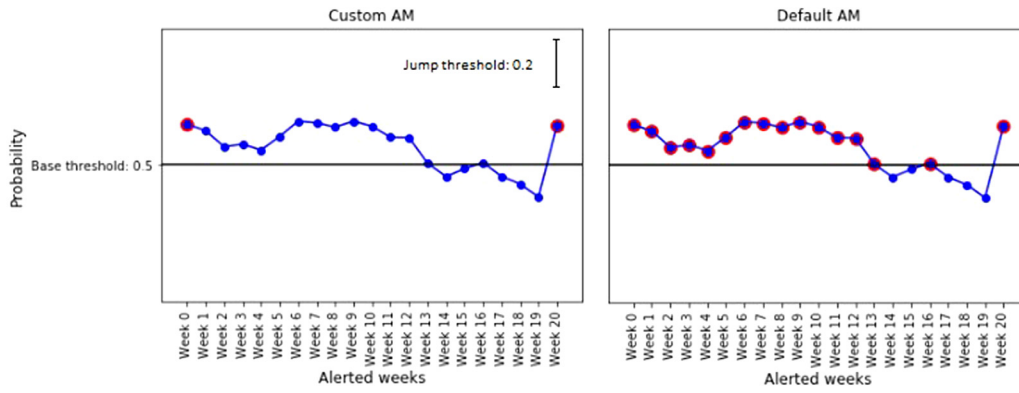


Fig. 7. Example of alerts raised by MSs for a given sequence of monitoring model scores of a single customer. Custom MS (left plot) with base threshold = 0.5 and jump threshold = 0.2; and default MS (right plot) with thr=0.5. Blue dots are model output scores on each week of monitoring and red dots are weeks, when the alerts are raised. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

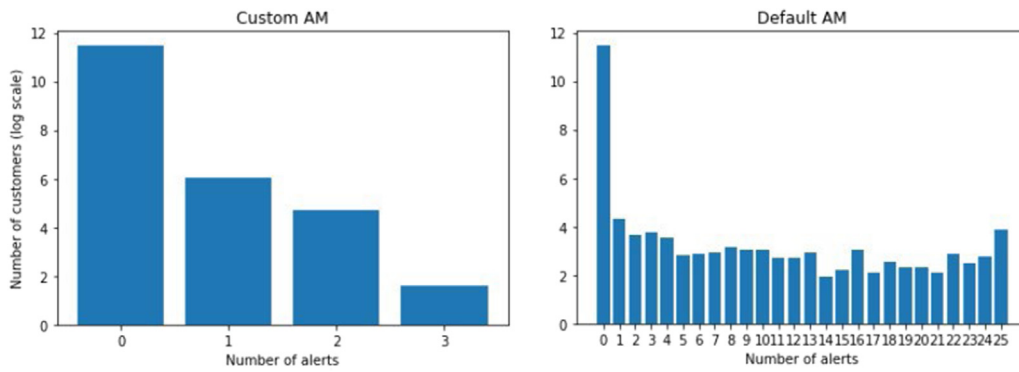


Fig. 8. Distribution of number of alerts raised per customer by custom MS (left plot) and default MS (right plot).

Table 1

Alert level CM (left) and entity level CM (right) for alerts raised by custom MS on customers from the test set.

Alert level CM				Entity level CM			
True	Predicted			True	Predicted		
		Class 0	Class 1			Class 0	Class 1
		1870981	277			97670	241
	Class 0	9856	381		Class 0	289	298
	Class 1				Class 1		
Recall = 0.037				Recall = 0.508			
Precision = 0.579				Precision = 0.553			
Custom recall = 0.433							
Custom precision = 0.401							

as $recall = \frac{TP}{TP+FN}$) is strikingly small. This was hypothesized earlier — if a reported customer got 1 alert in a test monitoring period of 25 weeks, then in the alert level CM we get +1 to TP and +24 to FN. And since recall should represent the proportion of reported customers that got alerted, we come to the conclusion that recall must be calculated on entity level. From Table 1 it can be seen that there were 277 alerts raised (alert level FP) on 241 randomly sampled customers (entity level FP). The precision (calculated as $precision = \frac{TP}{TP+FP}$) although it shows similar numbers for both alert level and entity level CMs, must be calculated on the alert level, since precision represents the proportion of correctly raised alerts for AML monitoring task. The end users of the monitoring system are AML domain experts which investigate raised alerts, and correctly calculated precision (precision on alert level) gives more correct assessment of the system's performance and helps in the estimation of human workload. To conclude, according to the business needs, precision of the AML MS has to be calculated on alert level while recall has to be calculated on entity level.

As stated earlier, one of the key requirements of monitoring setups is the timing of raised alerts. Very early alerts or very late alerts should

be considered as FPs. But CM and the standard definition of recall and precision do not have the capacity to reflect the timing aspect of raised alerts, thus those metrics misrepresent the actual performance of the MS. In order to provide not distorted performance estimation by taking into account timing aspect, we designed custom recall and custom precision (see Section 3.2). As explained above, the proposed custom recall is calculated at the entity level, and it counts as TPs only alerts raised close to a report date. Custom precision, in turn, also considers alerts raised close to a report date and penalizes early and late alerts.

For the custom MS with parameters defined above (base threshold = 0.5 and jump threshold = 0.2), we obtained a custom recall of 0.433 and a custom precision of 0.401. If we compare these values with the entity-based recall and alert-based precision (recall = 0.508 and precision = 0.579 as shown in Table 1) we see a substantial difference. This observation emphasizes the importance of taking into account the timing of alerts in the definition of performance measures to get the correct estimation of AML MS performance.

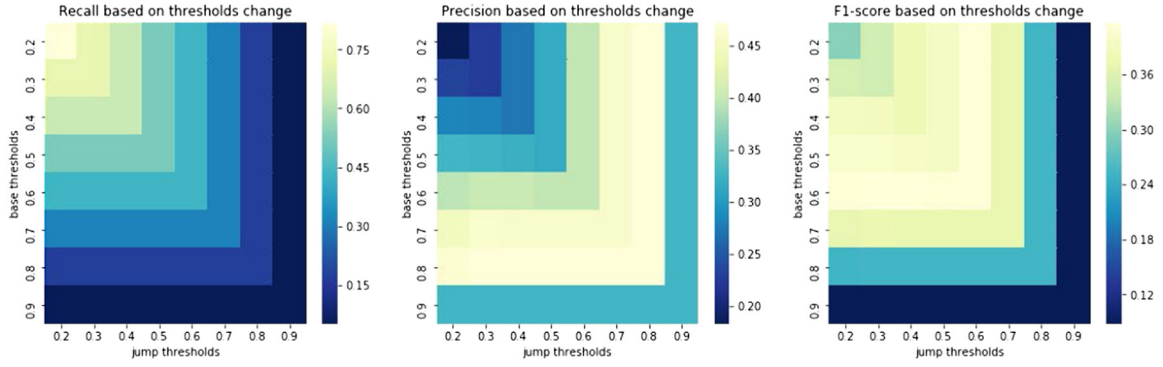


Fig. 9. Heat map of custom recall (left plot), custom precision (middle plot) and custom F1 (right plot) scores based on base threshold and jump threshold of the custom MS on a test set.

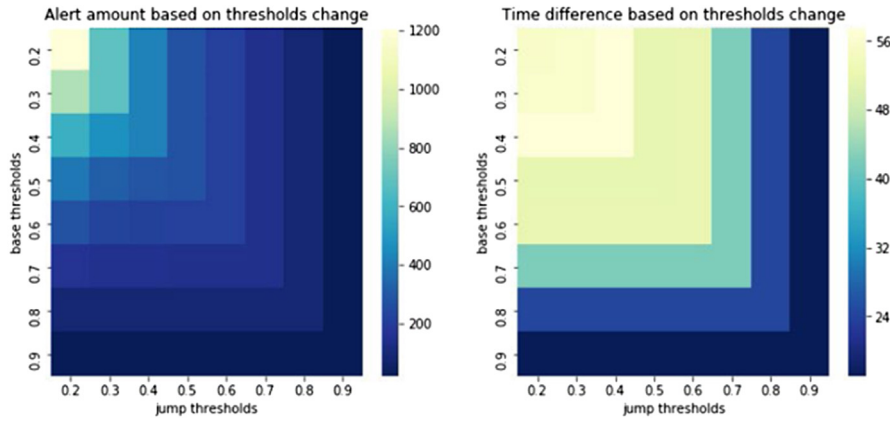


Fig. 10. Heat map of alerts amount (left plot) and average time between alert date and report date in days (right plot) based on base threshold and jump threshold for custom MS.

4.4. Tuning the custom MS (RQ3)

To analyse the behaviour of custom MS we made a grid search of parameters and calculated custom recalls, custom precisions and custom F1 scores (see Fig. 9).

We observe that the highest recall is achieved with the smallest possible threshold values. This is correct because with the smallest thresholds we will raise the maximum number of alerts, thus will catch the maximum number of customers with illicit behaviour. The highest precision is achieved when the base threshold and jump threshold are both in a range of 0.7–0.8 which shows that with very high thresholds there should be very limited number of alerts raised per customer, and they are raised in the most proper time. As of F1 score, the highest numbers are achieved when the base threshold and jump threshold are in a range of 0.5–0.6. At those ranges, the equilibrium between precision and recall is reached. Nevertheless, the final decision on parameters should be made by the subject-matter experts.

Since timing of alerts is crucial in the context of a monitoring system, we measured average time between report date and the date of the first alert raised for reported customers from the test set with different parameters of custom MS. Together with it, we measured number of raised alerts which directly defines the workload of the end users (see Fig. 10).

From the plot of time difference between an alert date and a report date, we can see that the parameters that maximize the F1-score (base threshold and jump threshold are both in a range of 0.5–0.6) has the average time difference of 45–50 days. This coincides well with the time period where we give the maximum utility for raised alerts — 6 weeks before to 2 weeks before the report.

This observation suggests that the F1 score is an appropriate measure for parameter threshold tuning. In the alerts amount heat-map, we

Table 2

Summary of samples testing rounds.

Round	Considered period	Number of alerts
1	01.2020–02.2020	24
2	02.2020–03.2020	24
3	03.2020–04.2020	24

observe that the lower the thresholds are, the more alerts are raised, as expected. The subject-matter experts may use such heat-maps to make a decision on the thresholds.

4.5. Evaluation with domain experts (RQ4 and RQ5)

The usefulness of the alert explanations was assessed via 3 testing rounds. In each of these testing rounds, we randomly selected 8 alerts raised by the custom MS for each of 3 countries covered in the study, leading to a total of 24 alerts per testing round (see Table 2). Each alert was analysed by an AML domain expert from the country in question. Based on this analysis, the investigators stated the relevance and accuracy of the alerts and the usefulness of the associated explanations. For the purpose of this experiment, we will focus solely on the assessment of the explanations. As explanations for the alerts, the subjects saw 5 of the most contributive features (top SHAP values) and 5 features which had changed the most (top delta SHAP values) digested in a human-readable representation of feature groups (FG). Each alert is represented with up to 10 FGs. More than one contributive feature can fall in the same group, e.g. 2 features, mean of incoming payments and sum of incoming payments fall into one FG “Payments patterns in incoming transactions”.

Table 3

Frequencies of times FGs were useful during 3 sample testing rounds.

Feature group	Round 1		Round 2		Round 3	
	# of times FG useful as most contributive characteristic in explanations	# of times FG useful as most changing characteristic in explanations	# of times FG useful as most contributive characteristic in explanations	# of times FG useful as most changing characteristic in explanations	# of times FG useful as most contributive characteristic in explanations	# of times FG useful as most changing characteristic in explanations
Transaction amounts patterns	18/24	8/10	23/24	3/3	19/23	0/0
Proportion of Foreign vs Domestic vs Intraday payments	7/13	1/3	13/14	1/3	17/18	0/0
Cash patterns in outgoing transactions	5/6	2/3	3/3	1/1	11/11	0/0
Cash patterns in incoming transactions	8/10	2/2	8/8	1/1	8/8	0/0
Payments patterns in outgoing transactions	0/0	1/1	0/0	0/0	1/1	0/0
Payments patterns in incoming transactions	6/10	3/5	14/14	4/4	13/18	0/0
Connection with previously reported customers	2/10	1/3	10/11	6/7	2/2	0/0
Channels usage patterns	5/11	1/1	2/2	1/1	10/10	0/0
Different currencies patterns	3/3	2/2	2/4	1/1	2/2	0/0
Smurfing patterns	2/6	3/5	1/1	2/2	2/2	0/0
Statistics and proportions of unique counterparties	0/0	1/1	0/0	0/0	0/0	0/0
Other account activity patterns (not cash nor payments)	1/3	1/2	2/3	0/0	1/4	0/0
Patterns of transaction values used (e.g. only few unique values or many round values)	0/0	0/1	0/0	0/0	0/0	0/0
Difference in aggregated sums of consecutive months	1/1	0/2	0/0	1/2	0/0	0/0
Counterparty country/bank patterns	0/0	1/2	0/0	0/0	0/0	0/0
Statistics of intertransaction times (time between consecutive transactions)	0/0	1/1	0/0	0/0	0/0	0/0

As explained above (in Section 3.4), if the alerts are raised on the very first week of monitoring then the explanation of the alert will consist only of the 5 most contributive features.

We took all the FGs that appeared during all 3 sample testing rounds and calculated frequencies of FGs found to be useful as most contributive (top SHAP) characteristic and as most changing (top delta SHAP) characteristic in explanations (see Table 3 for details). A value of 7/13 in this table means that during a testing round, a given FG appeared 13 times (for 13 different raised alerts) in the alerts explanations and 7 times it has been found useful by the domain experts. Some FGs did not appear in the explanations at all since they never happened to be the most contributive or the most changing ones.

We observe that some of the FGs are dominant and present almost in every alert, most notably “Transaction amount patterns” and “Cash patterns in outgoing transactions” while there are FGs which appeared only once for all three testing rounds, most notably “Statistics of intertransaction times”. The dominant FGs represent general customers’ behaviour, while dominated FGs represent specifics aspects of the behaviour.

Based on received feedback from AML domain experts after the first round (and subsequently after the second round), we improved the way we present the explanations. This can be noticed, for example, in the FG “Connection with previously reported customers”. This feature group was not found useful by the domain investigators in the first testing round because it was presented just as a fact that a customer has been in contact with known reported counterparties. It appeared that the initial explanation associated to this FG did not help the domain experts because it did not highlight the exact reported counterparties. We, therefore, updated the feedback to accompany the explanation of

each alert that referred to this FG with the counter-parties in question. After this update, starting from round 2, we notice an improvement in the perceived usefulness of this FG. Also, in the first round, some of the FGs were named in a way that domain experts found confusing and we re-formulated them for the second and third testing rounds. To conclude, based on the feedback from end users of the money laundering detection MS, the generated alerts explanations appear to be helpful to initiate investigations.

5. Conclusion

In this paper we presented a use case study of a predictive MS used in the context of AML monitoring in a multinational financial institution. The proposed MS addresses requirements of generating accurate, non-redundant and timely alerts. The underlying ML-based money laundering detection model is trained on snapshots of customers’ behaviour, with labels coming from historical reports sent on identified cases of money laundering. To generate not-redundant and timely alerts, the output scores of the ML model are passed to a designed MS which is raising alerts only when and as soon as customers’ behaviour change. Custom prediction quality metrics that take into account the reoccurring alerts and timing of the alerts generated by the MS are used. These custom quality metrics are used to optimize the MS parameters, hence ensuring that the optimized MS generates alerts close to the time when the corresponding investigation is relevant and prevents from raising redundant sequential alerts.

The presented predictive MS system is also designed to address ease-of-use requirements, specifically the requirement that the generated

alerts should be accompanied by explanations both related to the reason for the alert and the timing of the alert. The approach for generating explanations is based on Shapley values, but lifted at the level of high-level feature group descriptions, designed to help AML domain experts to start investigations.

The proposed method was designed in the context of a real-life industrial application in a multinational financial institution and evaluated using real-life data and with feedback from AML investigators. The evaluation showed that a standard baseline MS generates a significant number of redundant alerts, compared to the custom MS approach. It has been shown that according to AML business needs, recall should be calculated at the entity level, while the precision should be calculated at the alert level. In other words, false positives should be avoided at the alert level (false alerts), while false negatives should be avoided at the entity level (uncaught illicit customers). From the experiments it was clearly seen that standard recall (calculated on entity level) and precision (calculated on alert level) substantially over-estimate the actual performance of the MS, since they do not have the capacity to take into account reoccurring alerts and timing of the alerts aspects. In the opposite, custom metrics do take into account aforementioned AML aspects, thus giving the more correct assessment of the MS performance. In summary, the evaluation showed that a predictive MS optimized using the proposed custom metrics allows us to tackle the two issues at stake: timing of alerts and avoiding redundant reoccurring alerts.

The empirical evaluation also assessed the interpretability of the generated alerts via three testing rounds with AML domain experts. This evaluation showed that the investigators perceived that the explanations were useful as a starting point to investigate the corresponding alerts. As a future work for interpretability improvements, it is considered to apply a counterfactual explanations approach for alerts explanations generation. This approach may enrich generated textual explanations with "what if" type of statements which could highlight the minimal changes in customers behaviour needed to change the classification model decision.

The empirical evaluation with domain experts has a number of threats to validity and limitations. First, the evaluation has the typical ecological threats to validity associated to case study research, specifically limited generalizability due to the fact that the study was conducted in a single organization. However, we note that the evaluation involved three countries (and different domain experts from each country). Second, the empirical evaluation was made using a reduced number of alerts. A larger-scale evaluation is needed to draw more generalizable conclusions. Third, the evaluation focused on perceived relevance and perceived usefulness. A more in-depth evaluation of user acceptance in daily usage setting could lead to further insights into potential barriers to full adoption of MSs.

CRedit authorship contribution statement

Pavlo Tertychnyi: Conceptualization, Methodology, Software, Writing – original draft. **Mariia Godgildieva:** Methodology, Software. **Marlon Dumas:** Supervision, Writing – review & editing. **Madis Ollikainen:** Conceptualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was partly funded by ERDF via the NUTIKAS program and by the Estonian Research Council (grant PRG1226).

References

- Ahmed, A. A. A. (2021). Anti-money laundering recognition through the gradient boosting classifier. *Academy of Accounting and Financial Studies Journal*, 25(5).
- Alkhalili, M., Qutqut, M. H., & Almasalha, F. (2021). Investigation of applying machine learning for watch-list filtering in anti-money laundering. *IEEE Access*, 9, 18481–18496.
- Chao, X., Kou, G., Peng, Y., & Alsaadi, F. E. (2019). Behavior monitoring methods for trade-based money laundering integrating macro and micro prudential regulation: a case from China. *Technological and Economic Development of Economy*, 25(6), 1081–1096.
- Chen, Z., Soliman, W. M., Nazir, A., & Shorfuzzaman, M. (2021). Variational autoencoders and Wasserstein generative adversarial networks for improving the anti-money Laundering process. *IEEE Access*, 9, 83762–83785.
- Dachraoui, A., Bondu, A., & Cornuejols, A. (2015). Early classification of time series as a non myopic sequential decision making problem. In *Joint european conference on machine learning and knowledge discovery in databases* (pp. 433–447). Cham: Springer.
- Fawcett, T., & Provost, F. (1997). Adaptive fraud detection. *Data Mining and Knowledge Discovery*, 1(3), 291–316.
- FCE Network (2020). *History of anti-money laundering laws*. United States Department of the Treasury.
- He, G., Duan, Y., Peng, R., Jing, X., Qian, T., & Wang, L. (2015). Early classification on multivariate time series. *Neurocomputing*, 149, 777–787.
- Jullum, M., Løland, A., Huseby, R. B., Anonsen, G., & Lorentzen, J. (2020). Detecting money laundering transactions with machine learning. *Journal of Money Laundering Control*.
- Ketenci, U. G., Kurt, T., Önal, S., Erbil, C., Aktürkoglu, S., & İlhan, H. Ş (2021). A time-frequency based suspicious activity detection for anti-money laundering. *IEEE Access*, 9, 59957–59967.
- Kull, M., Silva Filho, T., & Flach, P. (2017). Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers. In *Artificial Intelligence and Statistics*. PMLR, Article 623631.
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Proceedings of the 31st international conference on neural information processing systems* (pp. 4768–4777).
- Mubalalike, A. M., & Adali, E. (2018). Deep learning approach for intelligent financial fraud detection system. In *2018 3rd international conference on computer science and engineering* (pp. 598–603). IEEE.
- Raiter, O. (2021). Applying supervised machine learning algorithms for fraud detection in anti-money laundering. *Journal of Modern Issues in Business Research*, 1(1), 14–26.
- Shapley, L. S. (1953). Contributions to the theory of games. In *Chapter a value for n-person games*.
- Sheridan, R. P. (2013). Time-split cross-validation as a method for estimating the goodness of prospective prediction. *Journal of Chemical Information and Modeling*, 53(4), 783–790.
- Sipos, R., Fradkin, D., Moerchen, F., & Wang, Z. (2014). Logbased predictive maintenance. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1867–1876).
- Tertychnyi, P., Slobozhan, I., Ollikainen, M., & Dumas, M. (2020). Scalable and imbalance-resistant machine learning models for anti-money laundering: A two-layered approach. In *Markets and services in the finance industry, International workshop on enterprise applications* (pp. 43–58). Cham: Springer.
- Xing, Z., Pei, J., Yu, P. S., & Wang, K. (2011). Extracting interpretable features for early classification on time series. In *Society for industrial and applied mathematics, Proceedings of the 2011 SIAM international conference on data mining* (pp. 247–258).
- Zhang, K., Xu, J., Min, M. R., Jiang, G., Pelechris, K., & Zhang, H. (2016). Automated IT system failure prediction: A deep learning approach. In *2016 IEEE international conference on big data* (pp. 1291–1300). IEEE.